Cloud Computing

Service Models and Economics

Lecture 02 - Part 1

Academic Year 2025/2026

DATA SCIENCE AND ARTIFICIAL INTELLIGENCE – University of Trieste

Learning Objectives

By the end of this lecture, you will be able to:

- Define cloud computing using NIST's essential characteristics
- Distinguish between IaaS, PaaS, SaaS, and FaaS service models
- Evaluate deployment models (public, private, hybrid, multi-cloud)
- Analyze cloud economics: CapEx vs OpEx trade-offs
- Calculate Total Cost of Ownership for cloud vs on-premise
- Apply FinOps principles for cloud cost management

What is Cloud Computing?

Before NIST: Many competing definitions

Common misconceptions:

- "It's just someone else's computer" – oversimplifies the model
- "It's virtualization" – virtualization is just one component
- "It's hosting" – misses self-service and elasticity

We need a precise definition to reason about cloud systems

Enter NIST (National Institute of Standards and Technology)

NIST Cloud Definition

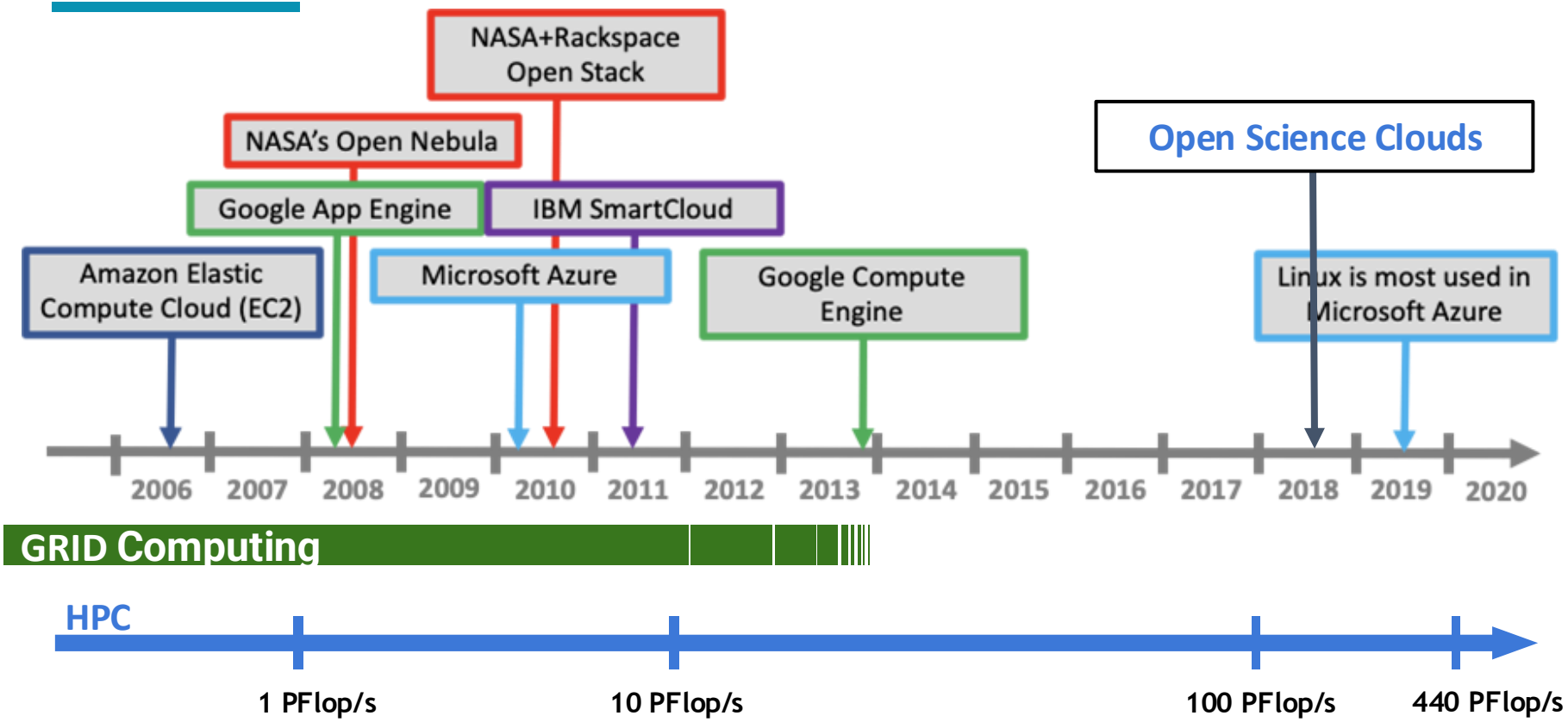
SP 800-145 (2011) – The standard reference

"Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort or service provider interaction."

NIST defines cloud through:

- 5 Essential Characteristics
- 3 Service Models (IaaS, PaaS, SaaS)
- 4 Deployment Models (Public, Private, Hybrid, Community)

Cloud Timeline



Part 1

Five Essential Characteristics

What makes cloud "cloud"

1. On-demand Self-service

A consumer can provision computing capabilities automatically without requiring human interaction with the service provider

What this means:

- No phone calls, no tickets, no waiting for approval
- Web console or API access 24/7
- Provision resources in minutes, not weeks

Example: Launch 100 VMs at 2am on a Sunday via API call

2. Broad Network Access

Capabilities are available over the network and accessed through standard mechanisms

What this means:

- Access from any device: laptop, phone, tablet, server
- Standard protocols: HTTPS, SSH, standard APIs
- No proprietary client software required

Implication: Network is required – cloud depends on connectivity

3. Resource Pooling

Provider's resources are pooled to serve multiple consumers using a multi-tenant model

What this means:

- Physical resources shared across many customers
- Location independence (customer doesn't control exact location)
- Statistical multiplexing enables efficiency

This is how providers achieve economies of scale → lower costs

4. Rapid Elasticity

Capabilities can be elastically provisioned and released to scale rapidly with demand

What this means:

- Scale UP: Add capacity when demand increases
- Scale DOWN: Remove capacity when demand decreases
- Appears unlimited to the consumer

Key differentiator from traditional hosting: scale in minutes, not months

5. Measured Service

Resource usage is monitored, controlled, and reported, providing transparency for provider and consumer

What this means:

- Pay-per-use: charged for actual consumption
- Metering: CPU hours, storage GB, network transfer
- Visibility: dashboards, billing reports, usage analytics

Enables the utility computing model: computing like electricity

Five Characteristics: Summary

1	**On-demand Self-service**	No human intervention
2	**Broad Network Access**	Access from anywhere
3	**Resource Pooling**	Multi-tenant, shared
4	**Rapid Elasticity**	Scale up/down quickly
5	**Measured Service**	Pay for what you use

If a service lacks any of these → it's not truly "cloud"

Part 2

Service Models

IaaS, PaaS, SaaS – The cloud stack

The Cloud Service Stack

Layers of abstraction and responsibility

SaaS	Software as a Service	Gmail, Salesforce, Office 365
PaaS	Platform as a Service	Heroku, App Engine, Elastic Beanstalk
IaaS	Infrastructure as a Service	EC2, Azure VMs, GCP Compute

↑ More abstraction, less control ↓ Less abstraction, more control

IaaS: Infrastructure as a Service

Provider manages: Physical hardware, networking, virtualization
You manage: OS, middleware, runtime, applications, data

What you get:

- Virtual machines
- Virtual networks
- Block storage
- Load balancers

Examples:

- AWS EC2
- Azure Virtual Machines
- Google Compute Engine
- DigitalOcean Droplets

Most control, most responsibility – closest to traditional IT

IaaS: Your Responsibilities

Applications	You manage
Data	You manage
Runtime	You manage
Middleware	You manage
Operating System	You manage
Virtualization	Provider manages
Servers	Provider manages
Storage	Provider manages
Networking	Provider manages

IaaS = You rent virtual hardware; you install and manage everything on top

PaaS: Platform as a Service

Provider manages: Hardware, OS, middleware, runtime
You manage: Applications and data only

What you get:

- Pre-configured runtime environments
- Automatic scaling
- Managed databases
- Development tools

Examples:

- Heroku
- Google App Engine
- AWS Elastic Beanstalk
- Azure App Service

Focus on code, not infrastructure – faster time to market

PaaS: Your Responsibilities

Applications	You manage
Data	You manage
Runtime	Provider manages
Middleware	Provider manages
Operating System	Provider manages
Virtualization	Provider manages
Servers	Provider manages
Storage	Provider manages
Networking	Provider manages

PaaS = You deploy your code; platform handles everything else

SaaS: Software as a Service

Provider manages: Everything – hardware through application
You manage: Your data and user configuration

What you get:

- Complete application
- Accessible via browser/API
- Automatic updates
- Multi-tenant by design

Examples:

- Gmail, Office 365
- Salesforce
- Slack, Zoom
- Dropbox, Google Drive

Least control, least responsibility – just use the application

Service Models: Comparison

Aspect	IaaS	PaaS	SaaS
You control	OS and up	App and data	Data only
Flexibility	Maximum	Medium	Minimum
Complexity	High	Medium	Low
Time to deploy	Hours/days	Minutes	Instant
Use case	Custom infrastructure	App development	End-user apps

Choose based on: How much control do you need vs. how much do you want to manage?

Part 3

Deployment Models

Where cloud runs: public, private, hybrid

Public Cloud

Infrastructure owned and operated by a third-party provider, available to the general public over the internet

Advantages:

- No upfront investment
- Virtually unlimited scale
- Provider handles maintenance

Considerations:

- Shared infrastructure
- Data sovereignty concerns
- Vendor lock-in risk

Examples: AWS, Microsoft Azure, Google Cloud Platform

Private Cloud

Infrastructure operated solely for a single organization, managed internally or by a third party

Advantages:

- Full control over data
- Customizable security
- Regulatory compliance easier

Considerations:

- Higher upfront costs
- Limited scale
- Requires expertise to manage

Examples: VMware vSphere, OpenStack, Azure Stack

Hybrid Cloud

Composition of two or more clouds (private, public)
that remain distinct but are bound together

Use cases:

- Cloud bursting: Use public cloud for peak demand overflow
- Sensitive data: Keep regulated data on-premise, other workloads in public cloud
- Disaster recovery: Backup to public cloud, run on private
- Migration: Gradual transition from on-premise to cloud

Challenge: Integration complexity – networking, security, management across environments

Multi-Cloud

Using services from multiple public cloud providers
(e.g., AWS + Azure + GCP)

Why multi-cloud?

- Avoid vendor lock-in
- Best-of-breed services
- Regulatory requirements
- Resilience (provider outage)

Challenges:

- Operational complexity
- Skills across platforms
- Data transfer costs
- Inconsistent tooling

Trend: Most large enterprises now use multi-cloud strategies

Deployment Models: Summary

Model	Infrastructure	Best For
Public	Provider-owned, shared	Startups, variable workloads, scale
Private	Organization-owned, dedicated	Regulated industries, sensitive data
Hybrid	Mix of public + private	Gradual migration, burst capacity
Multi-cloud	Multiple public providers	Avoid lock-in, best-of-breed

Part 2 covers: FaaS/Serverless, Cloud Architecture, Cloud Economics, Pricing Models, TCO, FinOps